

URI Escapist: um jogo digital 3D para aprendizagem em ambiente universitário

URI Escapist: a 3D Digital Game for Learning in a University Environment

Natã Cezer Bordignon¹, Daniel Menin Tortelli¹

¹ Universidade Regional Integrada do Alto Uruguai e das Missões
Departamento de Engenharias e Ciência da Computação
Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

103574@aluno.uricer.edu.br, danielmenintortelli@uricer.edu.br

Abstract. Introduction: Digital games can support educational practices by combining challenges, immediate feedback and active participation. In this context, URI Escapist is a three-dimensional game inspired by URI Erechim Campus II, in which the university environment is transformed into a setting for exploration, suspense and academic challenges. **Objective:** This work aims to develop an interactive experience that combines entertainment and learning through multiple-choice questions, exploration and progression mechanics. **Methodology or Steps:** The research has an applied and qualitative approach and adopts an iterative and incremental development process. The prototype was implemented in Unity and integrated with a PHP and MySQL web system through a REST application programming interface. **Results:** The current version includes two playable floors, three difficulty profiles, first-person exploration, stamina and exhaustion effects, contextual audio, runtime-generated interfaces and a teacher content-management panel. Questions are randomly selected by difficulty and floor, without repeated identifiers across different floors in the same session. Functional checks cover progression, navigation, question loading and audiovisual state management. **Conclusion:** The implemented architecture demonstrates the technical feasibility of separating educational content management from the game while preserving its direct effect on gameplay. User studies are still required to evaluate engagement, usability and learning outcomes.

Keywords Serious games, Game-based learning, Educational games, Unity, URI Escapist.

Resumo. Introdução: Os jogos digitais podem apoiar práticas educacionais ao combinar desafios, feedback imediato e participação ativa. Nesse contexto, o URI Escapist é um jogo tridimensional inspirado na URI Erechim Campus II, no qual o ambiente universitário é transformado em um cenário de exploração, suspense e desafios acadêmicos. **Objetivo:** Este trabalho tem como objetivo desenvolver uma experiência interativa que una entretenimento e aprendizagem por meio de perguntas de múltipla escolha, exploração e mecânicas de progressão. **Metodologia ou Etapas:** A pesquisa possui natureza aplicada e abordagem qualitativa, adotando um processo de desenvolvimento iterativo e incremental. O protótipo foi implementado na Unity e integrado a um sistema Web em PHP e MySQL por meio de uma interface de programação de aplicações REST. **Resultados:** A versão atual inclui dois andares jogáveis, três perfis de dificuldade, exploração em primeira pessoa, resistência e efeitos de exaustão, áudio contextual, interfaces geradas por código e um painel de conteúdo para professores. As perguntas são sorteadas conforme a dificuldade e o andar, sem repetição de identificadores entre andares distintos da mesma partida. Verificações funcionais abrangem progressão, navegação, carregamento de perguntas e controle dos

estados audiovisuais. Conclusão: A arquitetura implementada demonstra a viabilidade técnica de separar a gestão do conteúdo educacional do jogo e, simultaneamente, preservar seus efeitos diretos na jogabilidade. Ainda são necessários estudos com usuários para avaliar engajamento, usabilidade e resultados de aprendizagem.

Palavras-Chave Jogos sérios, Aprendizagem baseada em jogos, Jogos educacionais, Unity, URI Escapist.

1. Introdução

Os jogos digitais consolidaram-se como uma importante forma de entretenimento e passaram a ocupar espaço também em áreas como educação, saúde e capacitação profissional. Quando desenvolvidos com uma finalidade que ultrapassa o entretenimento, esses sistemas podem ser classificados como jogos sérios, pois preservam características lúdicas enquanto buscam promover aprendizagem, treinamento ou conscientização [?, ?].

No contexto educacional, a aprendizagem baseada em jogos utiliza desafios, regras, recompensas, feedback e progressão para favorecer a participação ativa do estudante. Prensky [?] observa que as tecnologias interativas se aproximam da linguagem utilizada pelas novas gerações, enquanto Gee [?] destaca que os jogos permitem aprender pela exploração, pela tentativa e pelo erro. Essas características tornam o jogo digital uma possibilidade complementar aos métodos tradicionais de ensino.

É nesse cenário que se insere o URI Escapist, um jogo digital em ambiente 3D inspirado na Universidade Regional Integrada do Alto Uruguai e das Missões, URI Erechim Campus II. O projeto transforma elementos da vivência universitária em uma experiência de exploração e suspense. Durante a partida, o jogador percorre ambientes acadêmicos, encontra livros com perguntas de múltipla escolha e precisa responder aos desafios enquanto administra o tempo disponível e evita ser capturado por um inimigo.

A escolha do tema está relacionada ao interesse do autor pela área de jogos digitais e ao potencial desse tipo de aplicação para apresentar conteúdos de maneira diferente e divertida. Em vez de substituir práticas educacionais existentes, o projeto procura investigar como uma experiência lúdica pode atuar como recurso complementar, estimulando atenção, raciocínio, tomada de decisão e engajamento.

O objetivo geral deste trabalho é desenvolver um jogo digital inspirado no ambiente universitário que una entretenimento e aprendizagem. De forma específica, pretende-se construir cenários reconhecíveis, implementar mecânicas de exploração e progressão, apresentar conteúdos por meio de perguntas, desenvolver uma ambientação imersiva e demonstrar o potencial dos jogos como apoio ao processo de ensino e aprendizagem.

As principais contribuições são a criação de um jogo sério contextualizado na realidade universitária local, a integração das respostas acadêmicas ao comportamento do inimigo e à progressão da partida e a construção de uma infraestrutura que permite a professores organizar e publicar perguntas sem modificar o projeto da Unity. A proposta também contribui como experiência prática de desenvolvimento, reunindo programação, design de jogos, inteligência artificial, interfaces Web, banco de dados e integração entre sistemas.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta os trabalhos relacionados; a Seção 3 reúne os conceitos que fundamentam o projeto; a Seção 4 descreve a metodologia; a Seção 5 detalha o desenvolvimento e as decisões técnicas; a Seção 6 apresenta os resultados parciais e sua discussão; e a Seção 7 expõe as conclusões e os trabalhos futuros.

2. Trabalhos Relacionados

Estudos sobre jogos e estratégias lúdicas no ensino superior indicam resultados promissores, mas também ressaltam que os efeitos dependem do contexto, do desenho da atividade e da relação entre mecânica e conteúdo. Vlachopoulos e Makri [?] analisaram jogos e simulações no ensino superior e identificaram contribuições para aprendizagem, habilidades e envolvimento, embora tenham apontado diferenças entre métodos e formas de avaliação.

Dichev e Dicheva [?] realizaram uma revisão crítica sobre gamificação na educação. Os autores observaram benefícios relacionados à motivação e ao engajamento, mas destacaram que ainda existem limitações metodológicas e evidências insuficientes para generalizações. Essa observação reforça a necessidade de avaliar o URI Escapist não apenas pela presença de elementos de jogo, mas pela experiência produzida e pela maneira como as perguntas são incorporadas às regras.

Wang e Tahir [?] revisaram estudos sobre a plataforma Kahoot!, baseada em quizzes. Os resultados apontaram efeitos positivos frequentes na aprendizagem, na dinâmica da sala e na atitude dos estudantes. Entretanto, a plataforma concentra a experiência na apresentação e resposta das questões, enquanto o URI Escapist insere o quiz em um ambiente tridimensional explorável, no qual erros e acertos alteram a perseguição, a dificuldade e o acesso a novas áreas.

Din, Baig e Khan [?] atualizaram uma revisão sobre jogos sérios e identificaram questionários, entrevistas e comparações antes e depois da intervenção entre os procedimentos utilizados para avaliar essas aplicações. O estudo apoia a escolha de combinar observação, registros da partida e questionários na avaliação do protótipo.

A Tabela ?? sintetiza as relações entre esses trabalhos e a proposta atual.

Tabela 1. Comparação entre abordagens relacionadas e o URI Escapist

Trabalho	Abordagem	Contribuição principal	Diferença em relação ao URI Escapist
Vlachopoulos e Makri	Revisão de jogos e simulações no ensino superior	Síntese de resultados educacionais e formas de aplicação	Não desenvolve um jogo contextualizado em um campus específico
Dichev e Dicheva	Revisão crítica de gamificação educacional	Identificação de benefícios e limitações das evidências existentes	Analisa elementos gamificados em geral, enquanto o projeto desenvolve um jogo completo
Wang e Tahir	Revisão de uma plataforma de quizzes	Evidências sobre aprendizagem, interação e atitude dos estudantes	O quiz não está associado à exploração tridimensional e à perseguição por inimigo
Din, Baig e Khan	Revisão atualizada de jogos sérios	Tendências de aplicação e métodos de avaliação	Possui escopo amplo, sem foco no cotidiano universitário local
URI Escapist	Jogo sério 3D com exploração e suspense	Integra perguntas, progressão e dificuldade em um cenário inspirado na universidade	Propõe uma experiência contextualizada, com consequências imediatas dos acertos e erros

O diferencial pretendido pelo URI Escapist está na combinação entre conteúdo acadêmico, representação de um espaço universitário reconhecível e mecânicas de suspense. A pergunta não funciona apenas como uma atividade sobreposta ao jogo: seu resultado modifica o estado da partida e a ameaça enfrentada pelo jogador. Essa integração constitui a lacuna específica explorada pelo trabalho.

3. Fundamentação Teórica

3.1. Jogos sérios

O conceito de jogo sério descreve aplicações que utilizam os recursos dos jogos com um propósito educacional, social ou de treinamento explicitamente planejado [?]. Apesar de possuírem metas externas ao entretenimento, essas aplicações dependem de elementos como desafio, regras, interação e feedback para manter o interesse do jogador.

Michael e Chen [?] apontam que a atratividade dos jogos pode ser empregada para envolver usuários em tarefas relacionadas à aprendizagem e ao desenvolvimento de habilidades. Zyda [?], por sua vez, ressalta a combinação entre narrativa, simulação e objetivos práticos como um diferencial dos jogos sérios. Dessa maneira, a experiência virtual pode contribuir para competências e conhecimentos aplicáveis fora do jogo.

O URI Escapist aproxima-se desse conceito ao inserir perguntas acadêmicas em uma experiência de exploração. O conteúdo educacional não é apresentado isoladamente: responder às questões interfere diretamente na progressão, no comportamento do inimigo e nas condições para concluir a fase.

3.2. Aprendizagem baseada em jogos

A aprendizagem baseada em jogos, também conhecida como *Game-Based Learning*, utiliza jogos como parte do processo educacional. Diferentemente da simples inserção de pontos ou recompensas em uma atividade, essa abordagem integra o conteúdo às regras e decisões da experiência [?].

Os jogos oferecem um ambiente no qual o participante pode experimentar, receber feedback imediato e ajustar sua estratégia. Gee [?] relaciona essas características a uma aprendizagem ativa, na qual o jogador desenvolve conhecimento ao resolver problemas contextualizados. Gros [?] também destaca a importância do planejamento da experiência e da relação entre os objetivos pedagógicos e as ações realizadas no jogo.

No projeto proposto, os livros encontrados no mapa apresentam perguntas de múltipla escolha. O resultado da resposta é informado imediatamente e produz consequências na partida. O acerto contribui para o progresso, enquanto o erro eleva a pressão sobre o jogador, reforçando a integração entre conhecimento e jogabilidade.

3.3. Prototipação e desenvolvimento iterativo

A prototipação permite validar conceitos e mecânicas antes da conclusão de todas as partes de um jogo. Fullerton [?] apresenta essa prática como uma etapa central do design, pois possibilita observar como as regras funcionam quando utilizadas por jogadores. Salen e Zimmerman [?] acrescentam que a iteração é importante para equilibrar o sistema e aproximar a experiência do resultado pretendido.

Essa abordagem é especialmente adequada ao URI Escapist, uma vez que aspectos como velocidade do inimigo, duração da fase, quantidade de perguntas e resistência do jogador precisam ser avaliados em conjunto. O desenvolvimento em ciclos permite implementar uma mecânica, testá-la e ajustar seus parâmetros antes da inclusão de novas funcionalidades.

4. Metodologia

Este trabalho caracteriza-se como uma pesquisa de natureza aplicada, com abordagem predominantemente qualitativa e caráter exploratório, prático e projetual. Além da discussão teórica sobre jogos digitais na educação, busca-se produzir e analisar um protótipo funcional que represente a proposta.

Inicialmente, foi realizada uma pesquisa bibliográfica sobre jogos sérios, aprendizagem baseada em jogos, gamificação e desenvolvimento de jogos digitais. A revisão forneceu suporte para relacionar as escolhas de design aos conceitos discutidos na literatura, comparar o projeto com abordagens existentes e definir os critérios de avaliação do protótipo.

Na etapa de planejamento foram organizados o conceito, a narrativa, as mecânicas, a estrutura das fases e o fluxo de interação. O desenvolvimento seguiu uma abordagem iterativa e incremental, na qual as funcionalidades foram implementadas em módulos e verificadas ao longo do processo. Esse método possibilitou incorporar progressivamente a movimentação, o quiz, a inteligência artificial, a progressão entre cenas e, posteriormente, o painel Web, o banco de dados e a API.

Foram realizadas verificações funcionais durante a implementação para observar movimentação, colisões, transições, sistema de perguntas, contadores, cronômetro, comportamento do inimigo e fluxo de conteúdo entre painel, API e jogo. Em uma etapa posterior, serão conduzidas sessões de teste com participantes convidados, observando-se a compreensão das regras, as dificuldades encontradas e a percepção sobre a integração entre entretenimento e conteúdo educacional.

A coleta de dados ocorrerá por meio de registros das etapas de desenvolvimento, observação das sessões de uso, dados básicos da partida e aplicação de um questionário breve. Serão considerados aspectos como facilidade de interação, clareza das perguntas, nível de desafio, imersão, interesse e percepção de utilidade educacional. Os registros de conclusão da fase, quantidade de acertos, erros e tempo utilizado complementarão a interpretação qualitativa.

5. Desenvolvimento do URI Escapist

5.1. Conceito, ambientação e estado do protótipo

O URI Escapist transforma o ambiente universitário em um espaço de exploração, suspense e desafios acadêmicos. Seus cenários são inspirados no início do Campus II da URI Erechim, na entrada do Prédio 8 e em seus andares. A identificação com um espaço conhecido é uma intenção de design, cuja efetividade ainda precisa ser investigada com participantes.

A direção visual combina modelos tridimensionais, texturas e efeitos inspirados na estética do primeiro PlayStation. Neblina, pixelização e processamento de imagem contribuem para a aparência retrô. Um filtro VHS acrescenta ruído, linhas de varredura, instabilidade horizontal e separação cromática. Esses elementos são recursos de apresentação, não indicadores de eficácia educacional.

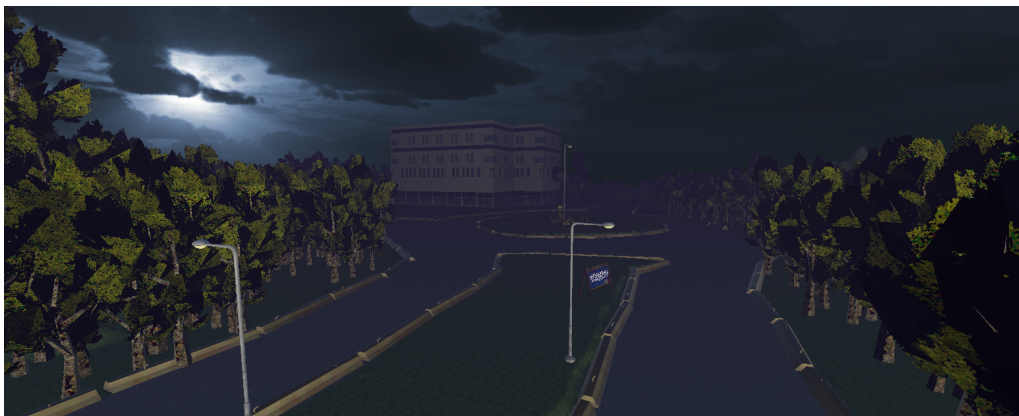


Figura 1. Cenário do URI Escapist inspirado na URI Erechim Campus II. Fonte: elaborado pelo autor.

A versão documentada contém o menu principal, a entrada e dois andares jogáveis. O terceiro andar ainda não foi implementado como fase disponível, embora sua regra de seleção de perguntas esteja prevista no servidor. Não há seleção livre de andar no menu atual: a progressão depende do cumprimento dos objetivos da partida.

5.2. Arquitetura e separação de responsabilidades

A solução é composta por um cliente tridimensional desenvolvido na Unity e um sistema Web responsável pela autoria e publicação do conteúdo. No cliente, componentes C# separam movimentação, câmera, resistência, perguntas, pontuação, navegação do inimigo, portas, interfaces e áudio. No servidor, PHP processa a autenticação e as operações administrativas, enquanto o MySQL persiste os registros. A API de leitura estabelece a comunicação por HTTP e JSON.

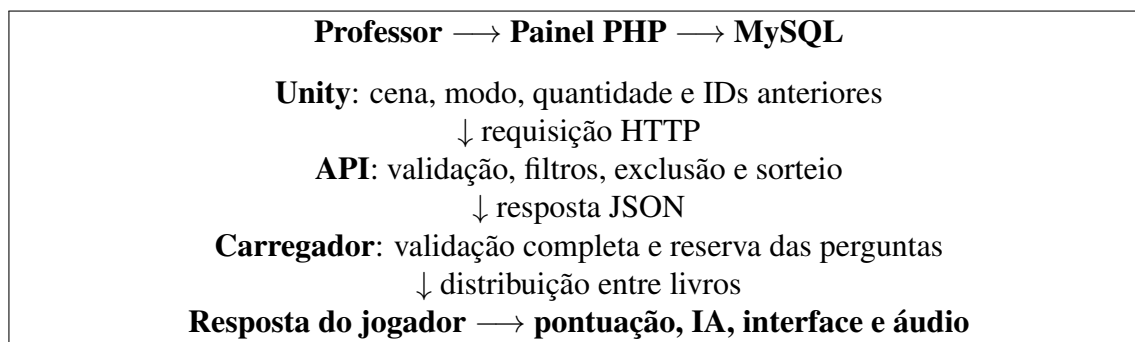


Figura 2. Fluxo integrado entre autoria, seleção de perguntas e execução do jogo. Fonte: elaborado pelo autor.

O componente `BookManager` controla os contadores e a disponibilidade das perguntas; `RemoteQuestionLoader` realiza a comunicação e mantém o histórico da sessão; `BookQuiz` conserva o conteúdo de cada livro; e `QuizManager` coordena abertura, fechamento e resposta. A classe `GameDifficulty` centraliza limites e multiplicadores dos três modos, evitando regras diferentes entre portas, contadores e comportamento do inimigo.

A apresentação é dividida entre `MenuPrincipal`, `GameInterface` e `GameHud`. O controlador de interface concentra estados modais, controle do cursor e suspensão do tempo. O áudio utiliza `GameAudio` e uma biblioteca de referências do tipo *ScriptableObject*. Os serviços de interface e áudio persistem entre cenas, enquanto os componentes da fase são novamente associados no carregamento. A sessão de perguntas é mantida em memória; não corresponde a um sistema de salvamento permanente.

5.3. Movimentação, câmera e resistência

O jogador explora o cenário em primeira pessoa, utilizando teclado e mouse. A movimentação emprega `Rigidbody`, colisores e verificações de contato com o chão. A orientação horizontal da câmera fornece a direção do deslocamento. A corrida utiliza a tecla Shift e consome resistência, apresentada como fôlego no HUD. Quando esse recurso termina, o controlador principal impede a corrida até sua recuperação completa.

O controlador de resistência possui parâmetros de capacidade, consumo, regeneração e atraso antes da recuperação. Seus valores padrão são 100 unidades de capacidade, consumo de 20 unidades por segundo, recuperação de 10 unidades por segundo e atraso de dois segundos. Esses valores são configuráveis no Inspector e não devem ser interpretados como medidas de esforço fisiológico real.

A rotação da câmera utiliza deslocamentos brutos do mouse, preservando a sensibilidade configurada sem multiplicá-los novamente pelo tempo do quadro. A rotação é aplicada em `LateUpdate`, e o primeiro deslocamento após recuperar o foco ou recapturar o cursor é desconsiderado. Essa alteração procura evitar saltos e diferenças de resposta associados à taxa de quadros. O balanço de caminhada e a respiração em repouso são tratados separadamente da orientação usada para mirar e caminhar.

5.4. Perfis de dificuldade e progressão

O menu oferece Fácil, Normal e Difícil. A seleção altera simultaneamente a quantidade de acertos necessária, o limite de erros, a composição das perguntas, os parâmetros do inimigo e o comportamento da leitura. O nível da questão, denominado fácil, média ou difícil no banco, é distinto do modo global. Assim, uma partida Normal pode começar com perguntas fáceis.

Tabela 2. Regras de progressão e leitura por dificuldade

Modo	Acertos	Derrota	Mundo na leitura	Fechar sem responder
Fácil	5	5º erro	Pausado	Permitido
Normal	7	3º erro	Em execução	Permitido
Difícil	9	1º erro	Em execução	Não permitido

Os limites da Tabela ?? são aplicados por andar. A derrota ocorre no erro que alcança o limite, não em uma tentativa posterior. As portas consultam o mesmo gerenciador utilizado pelo HUD, de modo que a meta mostrada corresponde ao requisito efetivo. Os contadores reiniciam ao carregar o andar, inclusive ao tentar novamente, enquanto a dificuldade escolhida permanece na sessão.

No fluxo atual, o jogador passa da entrada ao primeiro andar, cumpre a meta para alcançar o segundo e, após concluí-lo, retorna ao primeiro em direção à saída principal. A porta final exige o registro de conclusão do segundo andar; somente visitá-lo não libera a vitória. Uma nova partida limpa os registros de conclusão e a seleção anterior de perguntas. O terceiro andar não é necessário para concluir a versão atual.

A captura e o alcance do limite de erros levam à derrota. O cronômetro acompanha o tempo disponível da fase; seu esgotamento aciona a sequência de perseguição final existente no protótipo, que culmina na derrota. A interface permite tentar novamente no mesmo andar, iniciar outra partida após vencer ou retornar ao menu.

5.5. Inteligência artificial e crescimento da ameaça

O inimigo utiliza NavMeshAgent para calcular trajetórias sobre áreas navegáveis e percorrer corredores. A detecção considera alcance e linha de visão, verificada por raios físicos que desconsideram gatilhos e os próprios personagens. Quando a máscara de obstáculos está vazia, o código utiliza as camadas físicas padrão, evitando interpretar a ausência de configuração como visão através das paredes.

Tabela 3. Configuração da ameaça por modo de jogo

Modo	Velocidade base	Incrementos	Comportamento
Fácil	0,65×	0,25×	Patrulha; persegue ao detectar o jogador e volta à busca ao perder a detecção.
Normal	1,00×	1,00×	Segue devagar sem detecção e acelera quando detecta o jogador.
Difícil	1,50×	2,50×	Segue continuamente em velocidade de perseguição, mesmo sem detecção visual.

Os multiplicadores incidem sobre valores configurados no componente e incrementos produzidos pelas respostas. A patrulha do modo Fácil utiliza pontos definidos no cenário. Quando esses pontos não existem, como na configuração consultada do primeiro andar, o sistema sorteia destinos próximos, verifica sua presença no NavMesh e exige um caminho completo. Essa busca não utiliza a posição do jogador para escolher o destino.

O crescimento da velocidade é cumulativo e pode ser expresso, para cada velocidade de referência, por:

$$v = s [m_v v_0 + m_i (ca + eb + \lfloor c/2 \rfloor d)], \quad (1)$$

na qual v_0 é a velocidade inicial de patrulha ou perseguição, s é o fator de escala habilitado na cena, m_v e m_i são os multiplicadores do modo, c e e são acertos e erros, e a , b e d são os incrementos por acerto, erro e bônus a cada dois acertos. Os padrões do gerenciador são $a = 0,08$, $b = 0,12$ e $d = 0,04$. Um acerto posterior não elimina o aumento acumulado por erros.

A antiga ativação de perseguição permanente após erros sucessivos foi substituída pelos perfis da Tabela ???. Errar altera contadores e velocidade, mas não converte silenciosamente o Fácil em perseguição constante. Trata-se de adaptação determinística por regras, sem treinamento de modelos de aprendizagem de máquina ou personalização automática ao estudante.



Figura 3. Representação do inimigo no cenário do protótipo. Fonte: elaborado pelo autor.

5.6. Livros, resposta e retomada de perguntas

A aproximação de um livro por seu gatilho abre uma questão com quatro alternativas. A interface bloqueia movimentação e rotação controlada pelo mouse para permitir a seleção; a suspensão do mundo depende do modo. No Fácil, pergunta e resultado suspendem o tempo. No Normal e no Difícil, inimigo e cronômetro continuam ativos durante a interação.

No Fácil e no Normal, o jogador pode fechar a questão por Escape ou pelo botão correspondente. Essa ação não pontua, não registra erro, não remove o livro e não solicita uma pergunta diferente. O conteúdo permanece no objeto e pode ser retomado com E nas proximidades ou por nova entrada em sua área de interação. No Difícil, fechamento e abertura da pausa durante a pergunta são bloqueados; é necessário responder, salvo quando a partida termina por outra condição.

Após a escolha, o sistema aceita uma única resposta, informa acerto ou erro e apresenta a alternativa correta. O livro é retirado da cena, os contadores são atualizados e os parâmetros do inimigo são recalculados. A proteção contra respostas duplicadas impede pontuação adicional por cliques sucessivos. Se a resposta causar derrota, o estado final prevalece sobre a tela de feedback.



Figura 4. Registro da interface de perguntas do protótipo; os controles de fechamento variam conforme a dificuldade. Fonte: elaborado pelo autor.

5.7. Seleção de conteúdo sem repetição entre andares

A API utiliza modo e andar para definir categorias elegíveis, conforme a Tabela ???. O conteúdo é compartilhado, sem depender das associações antigas de cada pergunta a um piso específico.

Tabela 4. Dificuldade das perguntas por modo e andar

Modo	Primeiro andar	Segundo andar	Terceiro previsto
Fácil	Fáceis	Fáceis e médias	Médias
Normal	Fáceis	Médias	Difíceis
Difícil	Difíceis	Difíceis	Difíceis

O servidor organiza conjuntos por dificuldade, embaralha registros e alterna a retirada entre categorias quando há combinação. No segundo andar Fácil, procura-se equilibrar perguntas fáceis e médias disponíveis. Caso uma categoria seja esgotada pela exclusão de perguntas anteriores, a outra categoria permitida pode completar o conjunto. O cliente também embaralha os livros antes

da atribuição. A posição dos objetos permanece fixa: o que varia é o conteúdo associado a cada livro.

Para impedir repetições entre andares, o cliente reserva todos os IDs distribuídos na sessão, inclusive os dos livros não respondidos, e os envia no parâmetro `exclude`. O servidor valida a lista e utiliza parâmetros vinculados em uma exclusão SQL antes do sorteio e do limite de resultados. O cliente também rejeita respostas que reutilizem identificadores reservados, protegendo o fluxo contra servidores desatualizados.

O conjunto de cada andar é mantido em memória por modo e cena. Reiniciar ou revisitar esse mesmo andar reutiliza seu conjunto, com nova distribuição entre livros, sem consumir conteúdo destinado aos próximos pisos. Uma nova partida limpa o histórico e permite novo sorteio. A garantia é de não repetição de IDs entre andares distintos da sessão; não há detecção semântica de enunciados semelhantes cadastrados com IDs diferentes.

5.8. Banco inicial de conteúdo acadêmico

O repositório inclui uma carga de 90 perguntas de Ciência da Computação: 30 fáceis, 30 médias e 30 difíceis. As dez áreas contempladas são programação, algoritmos, estruturas de dados, banco de dados, redes de computadores, sistemas operacionais, arquitetura de computadores, segurança da informação, engenharia de software e fundamentos da computação. Cada área possui três perguntas de cada dificuldade.

As fáceis priorizam conceitos básicos; as médias incluem aplicação e interpretação; e as difíceis envolvem análise de situações, concorrência, complexidade e conceitos teóricos. Essa classificação inicial está sujeita à revisão docente e à avaliação empírica. A existência dos itens no repositório não comprova que todos tenham sido importados e publicados no servidor em execução.

A carga é armazenada em JSON e importada por um script PHP, que identifica o professor, cria disciplinas ausentes e publica as perguntas em uma transação. Uma nova execução ignora registros com o mesmo enunciado e professor, preservando edições anteriores. Há uma opção de validação sem gravação. Se faltarem questões inéditas elegíveis para preencher todos os livros, o jogo informa a falha em vez de repetir perguntas ou substituir categorias silenciosamente.

5.9. Interfaces, identidade visual e configurações

As telas são construídas em tempo de execução com Canvas, Unity UI, TextMeshPro e eventos de interface. O menu utiliza fundo escuro, tipografia serifada Cinzel, emblema procedural e destaque de seleção em azul associado à identidade visual da URI. A paleta azul substituiu a versão inicial dourada. A fonte Cinzel é acompanhada de sua licença SIL Open Font License no projeto. O CanvasScaler adequa a composição às resoluções, com referência de 1920 por 1080 pixels.

O menu principal apresenta nova partida, dificuldade, configurações, créditos e saída. As demais telas contemplam pausa, confirmação de reinício ou retorno, perguntas, resultado da resposta, derrota, vitória, carregamento e falhas da API. Os botões recebem interação por mouse e teclado. Concentrar o controle de cursor e tempo em `GameInterface` evita que componentes concorram pela captura do mouse ou deixem o mundo pausado após uma transição.

O HUD apresenta livros, erros e fôlego em cartões compactos com a mesma linguagem visual, além do cronômetro. Foram removidos os textos de depuração de stamina, corrida e exaustão. O aviso fixo de objetivo na parte inferior também foi retirado: o progresso é comunicado por uma mensagem de quatro segundos após a resposta, visível ao retornar ao jogo. As portas exibem avisos específicos para passagem bloqueada, avanço, retorno e saída final.



Figura 5. Registro da interface em uma etapa do protótipo. O HUD atual utiliza cartões compactos em azul e avisos temporários. Fonte: elaborado pelo autor.

As configurações permitem modificar volume geral e sensibilidade do mouse. O volume é aplicado por `AudioListener.volume`, e a sensibilidade multiplica a configuração base da câmera entre $0,2\times$ e $3\times$. As preferências são mantidas por `PlayerPrefs`. Esse armazenamento local destina-se a ajustes de uso, não à persistência de progresso acadêmico ou respostas da partida.

5.10. Efeito visual de exaustão

Ao atingir zero de resistência, o jogador recebe um efeito de tontura composto por oscilação suave da câmera, variação periódica do campo de visão e aumento da separação cromática no filtro VHS existente. O efeito não é disparado apenas por estar com pouco fôlego: exige uma condição explícita de esgotamento e diminui conforme o recurso se recupera.

O controlador utiliza funções senoidais e aproximação gradual da intensidade. Os deslocamentos são aplicados sobre a orientação normal da câmera sem alterar a direção de movimento do personagem. O deslocamento anterior de campo de visão é removido antes de aplicar o seguinte, evitando acúmulo. Livros, menus, perda de foco e desativação da câmera restauram os parâmetros visuais. A intensidade é configurável no Inspector, inclusive com valor zero para desativação.

A separação cromática é modulada em tempo de execução por `VHSFilterSettings`, sem sobrescrever os parâmetros originais do perfil. O processamento utiliza uma *renderer feature* da Universal Render Pipeline. A avaliação de conforto visual e acessibilidade ainda precisa ser realizada; o efeito constitui uma decisão de ambientação e sinalização de estado.

5.11. Áudio orientado aos eventos da partida

Foram integrados 11 arquivos WAV e MP3 em `Assets/sounds`. Uma biblioteca *ScriptableObject*, referenciada em `Resources`, associa arquivos e eventos e permite carregá-los também no executável. `GameAudio` mantém fontes separadas para ambiente, passos, perseguição, respiração, eventos ocasionais e interface, usando `AudioSource`, `AudioClip`, reprodução em ciclo e `PlayOneShot`.

Tabela 5. Associação dos arquivos de áudio aos eventos

Arquivo	Aplicação
som_clique	Botões dos menus e clique nos controles de configuração.
som_clique_correto	Resposta correta de um livro.
som_clique_errado	Resposta incorreta de um livro.
som_menus	Ambiente do menu principal.
som_ambiente_entrada	Ambiente da cena de entrada.
som_andares	Ambiente interno dos andares.
som_passos	Movimento no chão, com ritmo aumentado na corrida.
som_perseguiçao	Perseguição rápida do inimigo.
som_exausto	Recuperação após esgotamento do fôlego.
som_morte	Entrada na tela de derrota.
som_aleatorio	Evento nos andares a cada 25 a 45 segundos.

Os ambientes anteriores gerados por código deixam de executar quando a biblioteca está disponível, evitando sobreposição. O som de passos depende de deslocamento e contato com o chão; o de perseguição acompanha o estado rápido do inimigo. O ambiente é reduzido durante a perseguição para destacar a ameaça. O volume geral controla todas as fontes.

A pausa interrompe os sons do mundo e permite os cliques de interface. Nos modos em que o mundo continua ativo durante livros, o áudio acompanha essa continuidade. A derrota interrompe ambientes e ciclos, preservando o evento de morte, e a mudança de cena limpa as fontes anteriores. A configuração atual utiliza áudio não espacializado; posicionamento tridimensional e oclusão sonora são possibilidades futuras, não funcionalidades demonstradas nessa integração.

5.12. Sistema Web e modelo de dados

O painel permite que professores autenticados administrem disciplinas e perguntas sem modificar o executável. O acesso exige conta ativa, correio eletrônico e senha válidos. Indicadores apresentam totais de perguntas, publicações, rascunhos e disciplinas, além de alterações recentes. Disciplinas podem ser ativadas ou desativadas; a desativação preserva registros, mas retira suas perguntas do sorteio.

Cada pergunta contém enunciado, quatro alternativas, índice correto, dificuldade, publicação, disciplina e autoria. O servidor valida conteúdo, tamanho e consistência dos campos. A edição e a exclusão respeitam a autoria do professor autenticado. O estado de rascunho separa elaboração e disponibilização: somente perguntas publicadas de disciplinas ativas participam do sorteio.

A seleção de andar foi retirada do cadastro. Tabelas e associações antigas podem continuar no esquema por compatibilidade, mas não restringem o conteúdo entregue ao jogo. O andar orienta a regra de dificuldade pelo nome da cena; a pergunta permanece classificada por disciplina e dificuldade.

Tabela 6. Entidades e responsabilidades do banco de dados

Entidade	Responsabilidade
Professor	Identificação, autenticação e autoria das questões.
Disciplina	Organização temática e disponibilidade do conteúdo.
Andar	Nome, identificador textual, cena correspondente e estado ativo.
Pergunta	Enunciado, alternativas, gabarito, dificuldade e publicação.
Migração	Histórico de alterações incrementais do esquema.

A persistência utiliza MySQL, codificação UTF-8, chaves estrangeiras e índices de apoio às consultas. O acesso ocorre por PDO e consultas preparadas. As senhas são protegidas pelas funções de hash e verificação do PHP, e a autenticação renova o identificador de sessão. Operações de alteração utilizam POST e proteção CSRF, e a saída nas páginas é codificada. Essas medidas não equivalem a uma auditoria completa de segurança.

5.13. API, validação e tratamento de falhas

Ao carregar um andar, o jogo conta os livros ativos e solicita um lote do mesmo tamanho. A requisição a `/api/v1/questions.php` informa `scene`, `mode`, `limit`, `random` e `exclude`. O limite é controlado pelo servidor entre 1 e 50. A cena deve corresponder a um andar ativo, e a lista de exclusão aceita apenas identificadores inteiros positivos dentro dos limites definidos.

A resposta JSON contém IDs, enunciados, alternativas, gabaritos, dificuldades e informações das disciplinas, além de metadados de modo e cena. `UnityWebRequest` executa a consulta assíncrona com tempo máximo entre 2 e 30 segundos, e `JsonUtility` realiza a desserialização. Antes de preencher os livros, o cliente verifica quantidade exata, metadados, identificadores únicos e inéditos, quatro alternativas não vazias, índice correto e dificuldade permitida.

Lotes incompletos ou inválidos são rejeitados. Não há substituição por perguntas locais de dificuldade diferente como contingência. A prontidão permanece bloqueada e a interface apresenta a falha com opções de tentar novamente ou voltar ao menu. A insuficiência de perguntas inéditas produz HTTP 409; entradas inválidas recebem 422 e erros internos, 500. A rota de saúde verifica a comunicação com o banco, sinalizando disponibilidade ou falha.

O carregamento exibe progresso quando há informação disponível e um indicador indeterminado quando não é possível estimar o total transferido. As trocas de cena também são assíncronas. O mundo fica suspenso durante carregamento e erro para não penalizar o jogador pela indisponibilidade do serviço. Para um andar ainda não reservado, a conexão é necessária; reutilizar um conjunto em memória não constitui suporte geral a partidas desconectadas.

5.14. Tecnologias e ambiente de implantação

A Tabela ?? relaciona as tecnologias verificadas no projeto e suas aplicações. As versões descrevem as configurações consultadas no repositório, não a exigência de utilizar as versões mais recentes disponíveis.

Tabela 7. Tecnologias empregadas no projeto

Tecnologia ou recurso	Utilização
Unity 6000.0.58f2 e C# Rigidbody, colisores e Physics	Cenas 3D, componentes, regras e ciclo de execução. Movimento, contato com o chão, interação e obstáculos.
AI Navigation e NavMeshAgent	Áreas navegáveis, caminhos e deslocamento do inimigo.
Unity UI, TextMeshPro e EventSystem	Menus, HUD, alternativas e navegação por entrada.
URP, shaders e filtro VHS	Processamento de imagem e efeitos de exaustão.
AudioSource e AudioClip	Ambientes, ciclos e sons pontuais.
ScriptableObject e Resources	Biblioteca de referências aos sons do jogo.
PlayerPrefs	Persistência local de volume e sensibilidade.
UnityWebRequest e JsonUtility	Consulta HTTP e leitura das respostas JSON.
PHP 8.3, HTML, CSS e JavaScript	API, autenticação e interface administrativa Web.
PDO e MySQL 8.4	Consultas parametrizadas e persistência relacional.
Apache, Docker e Docker Compose	Servidor Web e orquestração dos serviços.
phpMyAdmin	Administração auxiliar do banco em desenvolvimento.
Git, Git LFS e GitHub	Histórico do código e tratamento de arquivos versionados.
LaTeX e BibTeX	Artigo, formatação SBC e referências bibliográficas.

A imagem da aplicação parte de PHP 8.3 com Apache e instala `pdo_mysql` e `mbstring`. O Docker Compose organiza aplicação, MySQL e phpMyAdmin, configurando rede interna, portas, variáveis de ambiente, verificações de saúde e volume persistente. A inicialização aplica esquema e migrações e permite criar uma conta inicial por configuração do ambiente.

O servidor pode executar em outro computador da rede local. O jogo deve utilizar uma URL acessível pelo endereço desse computador; `localhost` identifica a própria máquina do jogador. Mudanças no código da API exigem atualizar e reconstruir a imagem remota, enquanto alterações de conteúdo publicadas pelo painel não exigem recompilar o jogo. O desenvolvimento utiliza HTTP; uma implantação externa deve incorporar HTTPS, credenciais próprias e revisão dos acessos expostos.

5.15. Escopo e decisões de implementação

O escopo atual compreende exploração em primeira pessoa, corrida, exaustão audiovisual, dois andares, três modos, questões, progressão, derrota, tempo, interfaces completas, áudio contextual e conteúdo remoto. As revisões priorizaram consistência entre apresentação e regras: limites inclusivos de erros, portas ligadas à dificuldade, vitória condicionada à conclusão, exclusão de perguntas anteriores e controle centralizado de pausa.

Não fazem parte da versão demonstrada um terceiro andar jogável, multijogador, integração institucional, aplicativo móvel, realidade virtual, análise pedagógica automática ou adaptação por aprendizagem de máquina. O histórico de sessão não persiste após encerrar a aplicação. A posição dos livros não é sorteada, embora o conteúdo seja aleatório. Balanceamento, conforto visual e mixagem de áudio permanecem sujeitos a ajustes com testes de uso.

6. Resultados Parciais e Discussão

6.1. Resultados implementados

O resultado técnico é um protótipo integrado no qual o conteúdo acadêmico participa da progressão e da ameaça enfrentada pelo jogador. Os modos diferenciam pressão temporal e comportamento do inimigo, além da dificuldade das perguntas. Isso permite comparar leitura protegida no Fácil com tomada de decisão sob continuidade do mundo nos outros modos, sem pressupor que maior pressão resulte em melhor aprendizagem.

A autoria foi separada do cliente pelo painel e pela API. A carga inicial oferece 90 itens em dez áreas, e o sorteio mantém variedade entre andares sem alterar as cenas. O cache preserva consistência nas tentativas do mesmo piso, enquanto a exclusão de IDs impede reutilização de conteúdo distribuído em outros andares.

Tabela 8. Síntese dos resultados verificáveis na implementação

Área	Resultado atual
Progressão	Metas de 5, 7 e 9 acertos; derrota no 5º, 3º e 1º erro; saída condicionada à conclusão.
IA	Patrulha e perseguição por modo, visão e crescimento cumulativo de velocidade.
Perguntas	Sorteio, exclusão entre andares, cache e retomada sem consumir a questão.
Interface	Menus por código, HUD compacto, preferências e tratamento de falhas.
Audiovisual	Tontura por exaustão e 11 sons associados ao contexto.
Conteúdo Web	Autenticação, disciplinas, autoria, publicação e carga inicial.
Implantação	Contêineres, banco persistente e acesso pela rede local.

6.2. Verificações técnicas realizadas

A validação de desenvolvimento incluiu compilação C#, verificações dos endpoints PHP e execuções isoladas na Unity. Os testes da API utilizaram SQLite em memória como fixture, sem acessar o banco remoto. Esse recurso de teste não substitui o MySQL da aplicação. Foram verificadas combinações de modo e andar, aleatoriedade, limites, categorias, exclusões, parâmetros inválidos e insuficiência de conteúdo. A regra futura do terceiro andar foi exercitada em fixture, sem afirmar sua existência como fase concluída.

No cliente, verificaram-se metas imediatamente antes e no limite, derrota no erro exato, gatilhos de portas, bloqueio após término, conclusão e limpeza da sessão. Também foram conferidos fechamento e retomada do livro, preservação da pergunta, proteção contra resposta duplicada, pausa no Fácil e bloqueio de saída no Difícil. Uma execução isolada com NavMesh real e obstáculo físico verificou destinos, velocidades com detecção e oclusão, busca sem pontos de patrulha e crescimento cumulativo.

A câmera foi verificada com deslocamentos acumulados do mouse em diferentes partições de quadros e um quadro irregular, além de limites de rotação e recuperação de foco. Para exaustão, foram conferidos disparo em zero, entrada gradual, recuperação, ausência de reativação antes de novo esgotamento e restauração do campo de visão e das cores. No áudio, verificaram-se as 11 referências importadas, ambientes por cena, chamadas de reprodução, pausa, retomada e término.

A interface passou por inspeção de renderização e verificações de acomodação de texto em execução isolada. Esses procedimentos documentam propriedades funcionais e regressões específicas; não constituem medição de desempenho em diversos equipamentos, certificação de acessibilidade, auditoria de segurança ou comprovação da qualidade perceptiva de todos os efeitos. Os percursos dos mapas completos e o balanceamento final continuam exigindo testes integrados.

6.3. Discussão, limitações e avaliação prevista

Um acerto contribui para a passagem, mas também aumenta a velocidade do inimigo; um erro aproxima a derrota e modifica a ameaça. O modo define se há tempo protegido para refletir ou necessidade de responder com o mundo em movimento. A adequação pedagógica dessas condições deve ser discutida conforme público, conteúdo e finalidade da atividade.

A atualização de questões independente do executável introduz dependência do servidor para novos andares. Rejeitar lotes incompletos preserva a coerência das regras ao custo de impedir o início da fase até a correção. A ausência de repetição é limitada aos IDs da sessão: enunciados equivalentes cadastrados separadamente ainda exigem revisão editorial.

Não foram obtidas evidências de ganho de aprendizagem ou resultados de avaliação controlada com participantes nesta etapa. São necessárias sessões para investigar clareza das regras e perguntas, desafio, diversão, imersão, conforto audiovisual e utilidade educacional. Acertos, erros, tempo e conclusão poderão complementar observações e questionários, sem confundir desempenho sob pressão com domínio de conteúdo.

A avaliação poderá dialogar com os resultados sobre quizzes de Wang e Tahir [?] e com as limitações metodológicas discutidas por Dichev e Dicheva [?]. A classificação das questões deverá ser revisada por professores, e os parâmetros de dificuldade, ajustados com participantes. O registro persistente de telemetria, quando implementado, deverá prever minimização e anonimização dos dados.

7. Conclusão e Trabalhos Futuros

O URI Escapist materializa uma proposta de jogo sério contextualizada no ambiente universitário, reunindo exploração tridimensional, perguntas e suspense. A versão atual apresenta dois andares jogáveis, progressão e derrota coerentes com três perfis, perguntas sem repetição entre pisos da sessão, interfaces unificadas e feedback audiovisual associado aos eventos.

A integração entre Unity, PHP e MySQL separa autoria e execução. Professores podem organizar e publicar conteúdo; o cliente solicita conjuntos compatíveis com modo e fase, verifica sua integridade e informa problemas. O tratamento explícito de falhas substitui a contingência local, enquanto a memória de sessão mantém conjuntos em revisitas e tentativas do mesmo andar. A implantação em contêineres facilita reproduzir o ambiente Web e acessá-lo por outra máquina.

O trabalho reúne programação C#, física, navegação, interfaces, processamento de imagem, áudio, desenvolvimento Web, banco de dados e comunicação entre sistemas. As verificações sustentam a viabilidade técnica em cenários controlados, mas não permitem concluir que a experiência melhora a aprendizagem ou que a pressão do inimigo seja adequada a todos os participantes.

Como trabalhos futuros, propõem-se implementar o terceiro andar, ampliar e revisar as questões, avaliar o balanceamento nos mapas completos, oferecer controles de conforto audiovisual ao jogador, aprimorar transições e espacialização do áudio, estudar a randomização física dos livros e registrar partidas de forma anonimizada. Também permanecem previstas publicação por HTTPS, revisão dos mecanismos de acesso e avaliações com usuários, incluindo comparações anteriores e posteriores ao uso e, quando viável, grupos de comparação.